



Capstone Project

Understanding Knowledge Sources and Knowledge Bases for Agentic Retrieval





A knowledge source is the connection to the data. It tells the retrieval system where the information lives. That data could come from Blob Storage, an existing Azure AI Search index, Azure SQL, SharePoint, OneLake, the web, or even a remote MCP source.

A knowledge base is the retrieval-ready layer built from one or more knowledge sources. It organizes the grounded content so that an agent or chat application can search it and use the results when answering user questions.

In our capstone project, the main knowledge source is Azure Blob Storage. This storage account contains the CloudXeus course product sheets as PDF files. These PDFs provide the trusted business content that our agent will use later.

Once the knowledge source is connected, Azure AI Search can build the knowledge base from that content. The knowledge base becomes the place where the agent can retrieve relevant information during execution, instead of relying only on the model's general knowledge. The end-to-end idea is simple: connect to the data source, build a knowledge base from that data, and then allow the agent or chat application to query the knowledge base when it needs grounded information.



What Makes Agentic Retrieval Different?

How it goes beyond normal search and classic retrieval

 Normal Search / Classic Retrieval	 Agentic Retrieval
 Returns ranked documents or chunks	 Plans retrieval steps before answering
 Usually follows a single retrieval path	 Can retrieve from multiple chunks and sources
 Limited awareness of conversation context	 Uses conversation context to guide retrieval
 The developer often handles orchestration manually	 Can run retrievals in parallel to reduce latency
	 Better suited for complex, multi-part questions



Key idea: search finds content, but agentic retrieval plans how to gather the right grounded evidence.



Why this matters in our capstone: the agent must answer grounded product questions more intelligently than a basic search flow.



In a normal search flow, the system usually returns ranked documents or chunks. The search engine finds the content that appears most relevant to the user's query, and then the application or developer decides how to use those results. This is useful, but it often follows a single retrieval path and has limited awareness of the broader conversation.

Agentic retrieval goes a step further. Instead of only searching once and returning results, the agent can plan how to gather the right grounded evidence before answering. It can decide what information it needs, retrieve from multiple chunks or sources, and use the conversation context to guide the retrieval process.

This is especially useful when the user asks a complex or multi-part question. For example, a user may ask about course pricing, refund eligibility, product features, and delivery timelines in the same conversation. A basic search flow may return only one set of results, but an agentic retrieval flow can gather information from different sources and combine it into a more complete answer.

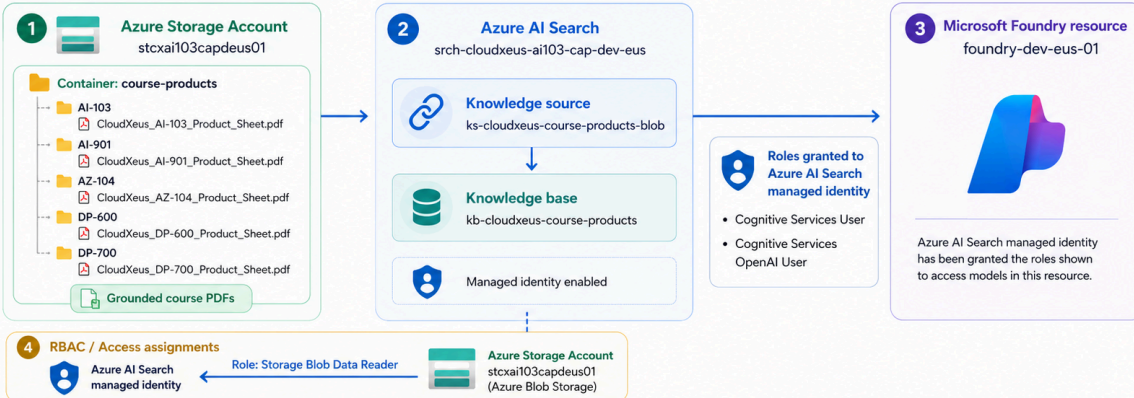
Another important benefit is orchestration. In classic retrieval, the developer often has to manually control the retrieval logic. With agentic retrieval, the agent can take on more of that decision-making. It can plan retrieval steps, call the right knowledge source, and use the retrieved content to produce a grounded response.

The key idea is that search finds content, but agentic retrieval plans how to gather the right evidence. This makes the final response more accurate, more contextual, and better suited for real business questions.

Knowledge source pre-requisites - Assigning permissions

Capstone Setup Overview

What we have configured so far for CloudXeus agentic retrieval



Why this matters: Azure AI Search can now read the CloudXeus PDFs, use Microsoft Foundry models for enrichment and retrieval, and build the grounded foundation for our agent.



The first part of the setup is the Azure Storage account. This is where the CloudXeus course product PDFs are stored inside the `course-products` container. Each course, such as AI-103, AI-901, AZ-104, DP-600, and DP-700, has its own product sheet. These PDFs become the grounded source documents for the agent.

Next, Azure AI Search is configured to use those documents. A knowledge source is created to connect Azure AI Search to the Blob Storage container. Then a knowledge base is created from that knowledge source. This knowledge base is what organizes the content so it can be searched and used during retrieval.

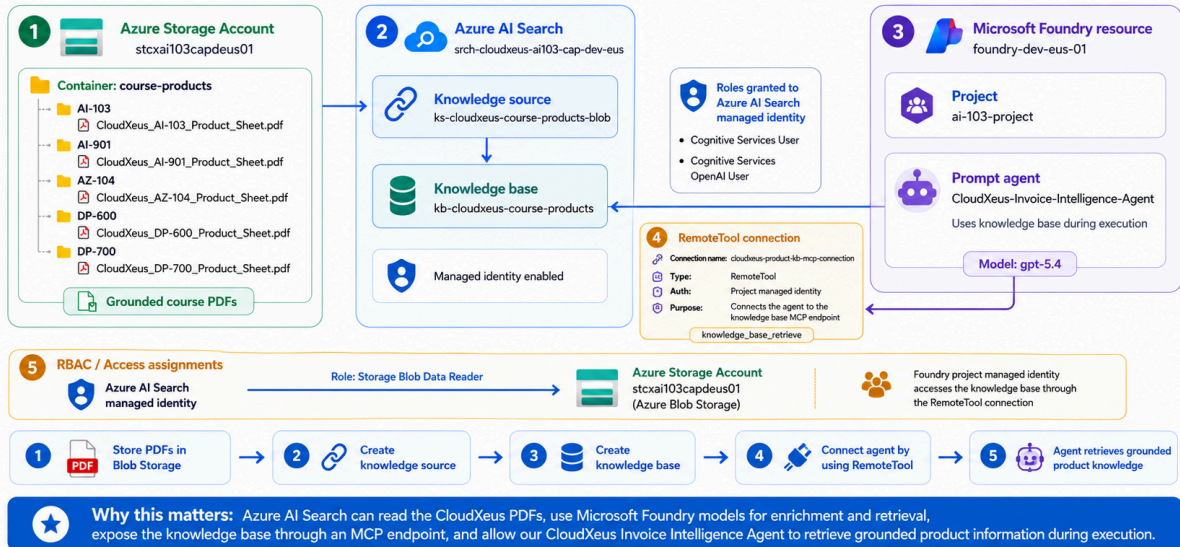
Managed identity is also enabled for Azure AI Search. This is important because Azure AI Search needs secure access to the storage account and to the Microsoft Foundry resource. Instead of using keys in the application code, we use role-based access control. For storage access, the Azure AI Search managed identity is granted the `Storage Blob Data Reader` role on the storage account. This allows Azure AI Search to read the PDFs from Blob Storage. For model access, roles such as `Cognitive Services User` and `Cognitive Services OpenAI User` are granted so Azure AI Search can use the required Foundry models for enrichment and retrieval.

The overall flow is: store the PDFs in Blob Storage, create a knowledge source, create a knowledge base, and then enable the required model and storage access using RBAC.

Building the CloudXeus Invoice Intelligence Agent - Remote Tool connection

Capstone Setup Overview

What we have configured so far for the CloudXeus agentic retrieval and agent workflow



The key connection between the agent and the knowledge base is the RemoteTool connection. This exposes the knowledge base retrieval capability to the agent through an MCP endpoint. When the agent needs product information, it can call the `knowledge_base_retrieve` tool and use the retrieved content as context for its response.

The end-to-end flow is: store the PDFs in Azure Blob Storage, create the knowledge source, create the knowledge base, connect the agent using RemoteTool, and allow the agent to retrieve grounded product knowledge during execution.